K Most Frequent Strings

Find the `k` most frequently occurring strings in an array, and return them sorted by frequency in descending order. If two strings have the same frequency, sort them in lexicographical order.

Example:

```
Input: strs = ['go', 'coding', 'byte', 'byte', 'go', 'interview', 'go'], k = 2
Output: ['go', 'byte']
```

Explanation: The strings "go" and "byte" appear the most frequently, with frequencies of 3 and 2, respectively.

Constraints:

- `k ≤ n`, where `n` denotes the length of the array.

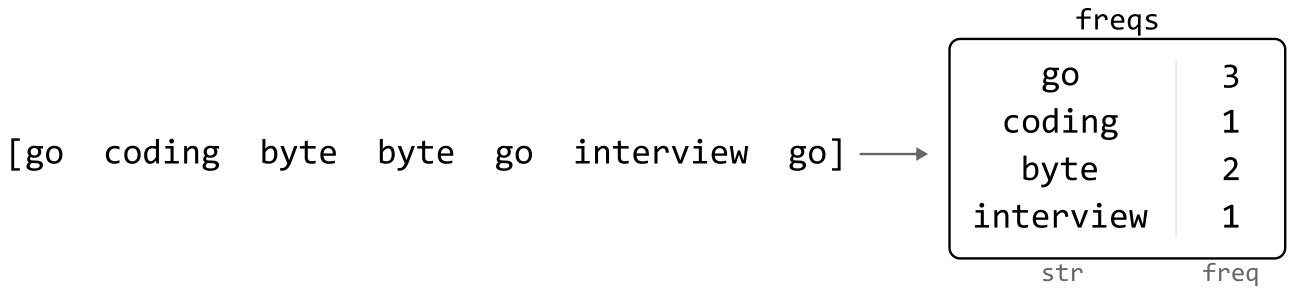
Intuition - Max-Heap

The two main challenges to this problem are:

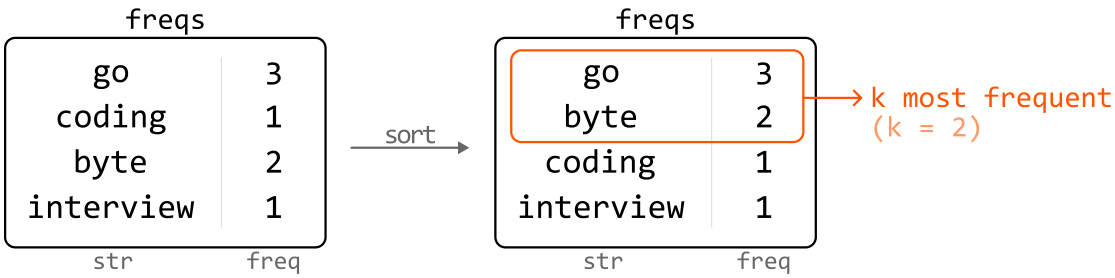
1. Identifying the k most frequent strings.
2. Sorting those strings first by frequency and then lexicographically.

For now, let's concentrate on identifying the most frequent strings and address lexicographical ordering afterward.

First, we need a way to keep track of the frequencies of each string. We can use a hash map for this, where the keys represent the strings and the values represent frequencies:



The most straightforward approach is to obtain an array containing the strings from the hash map, sorted by frequency in descending order. The k most frequent strings would be the first k strings in this array.



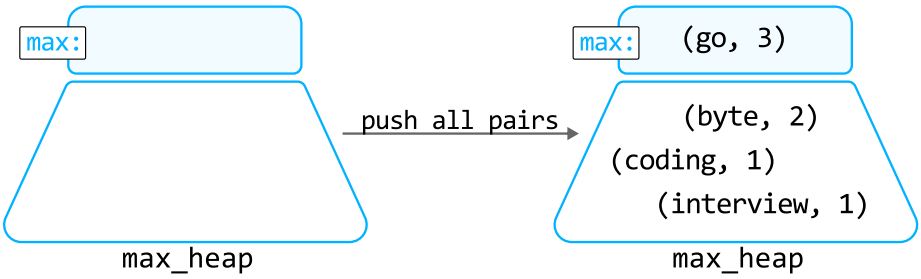
The main inefficiency of this solution is that it involves sorting all n strings, even though we only need the top k frequent ones to be sorted.

Something useful to consider: if we remove the most frequent string, the new most frequent string after this removal represents the second-most frequent overall. By repeatedly identifying and removing the most frequent string k times, we efficiently obtain our answer.

To implement this idea, we need a data structure that allows efficient access to the most frequent string at any time. A **max-heap** is perfect for this.

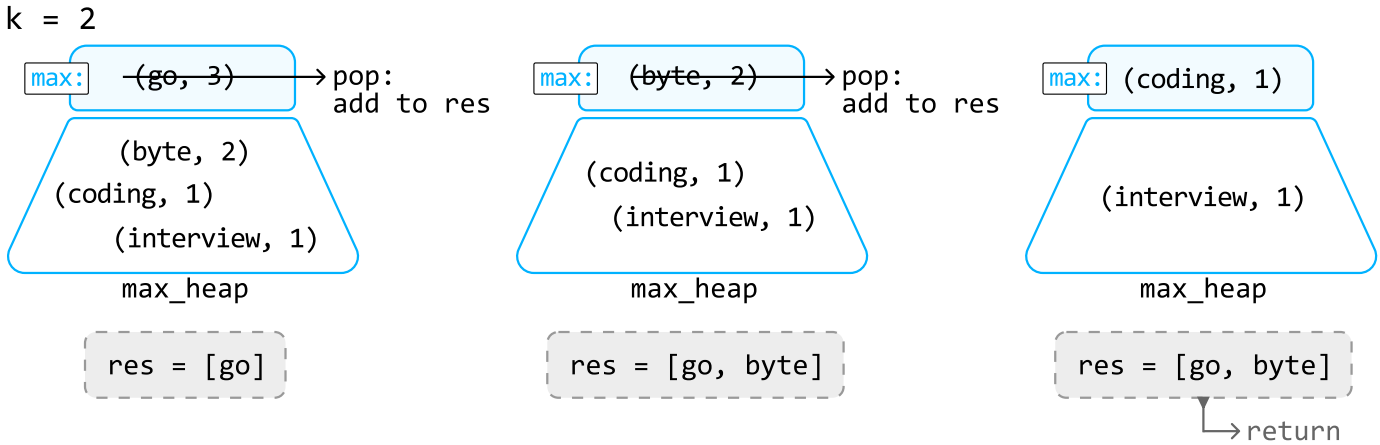
Max-heap

Let's find the k most frequent strings from the previous input, this time using a max-heap. First, populate the heap with each string along with their frequencies.



One way to populate the heap is to push all n strings into it one by one, which will take $O(n \log(n))$ time. Instead, we can perform the **heapify** operation on an array containing all the string-frequency pairs to create the max-heap in $O(n)$ time.

To collect the k most frequent strings, pop off the top element from the heap k times and store the corresponding strings in the output array `res`:



Now, we just need to ensure that when two strings have the same frequency, the one that comes first lexicographically has a higher priority in the heap. To do this, we can define a custom comparator for the heap that prioritizes strings lexicographically when their frequencies match, as demonstrated in the implementation below.

Try it yourself

Write your solution to the problem before checking the reference implementation.

Implementation - Max-Heap

We create a Pair class for string-frequency pairs, enabling us to customize priority using a custom comparator.

Python Java

```
import java.util.ArrayList;
import java.util.HashMap;
```

```
import java.util.Map;
import java.util.PriorityQueue;

class Pair {
    String str;
    int freq;

    public Pair(String str, int freq) {
        this.str = str;
        this.freq = freq;
    }
}

public class Main {
    public ArrayList<String> k_most_frequent_strin
        // We use a HashMap to count the frequency
        Map<String, Integer> freqs = new HashMap<>
        for (String s : strs) {
            freqs.put(s, freqs.getOrDefault(s, 0) +
        }
        // Create a max heap using a custom compar
        PriorityQueue<Pair> maxHeap = new Priority
            // Prioritize strings with higher freq
            if (a.freq != b.freq) {
                return b.freq - a.freq;
            }
            // If frequencies are equal, prioritiz
            return a.str.compareTo(b.str);
        });
        // Add all string-frequency pairs to the m
        for (Map.Entry<String, Integer> entry : fr
            maxHeap.offer(new Pair(entry.getKey(),
        }
        // Pop the most frequent strings off the h
        ArrayList<String> result = new ArrayList<>
        for (int i = 0; i < k && !maxHeap.isEmpty(
            result.add(maxHeap.poll().str);
        }
        return result;
    }
}
```

Complexity Analysis

Time complexity: The time complexity of `k_most_frequent_strings_max_heap` is $O(n + k \log(n))$.

- It takes $O(n)$ time to count the frequency of each string using `Counter`, and to build the `max_heap`.
- We also pop off the top of the heap k times, with each pop operation taking $O(\log(n))$ time.

Therefore, the overall time complexity is $O(n) + kO(\log(n)) = O(n + k\log(n))$.

Space complexity: The space complexity is $O(n)$ because the hash map and heap store at most n pairs. Note that the output array is not considered in the space complexity.

Intuition - Min-Heap

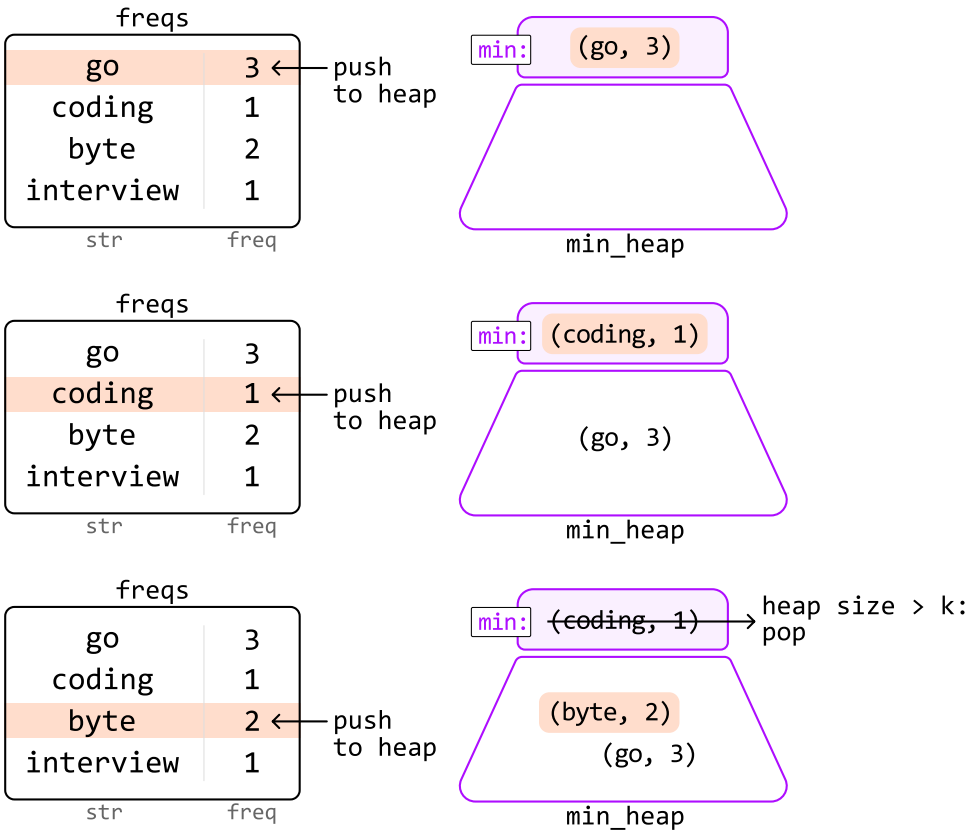
As a follow up, your interviewer may ask you to modify your solution to reduce the space used by the heap.

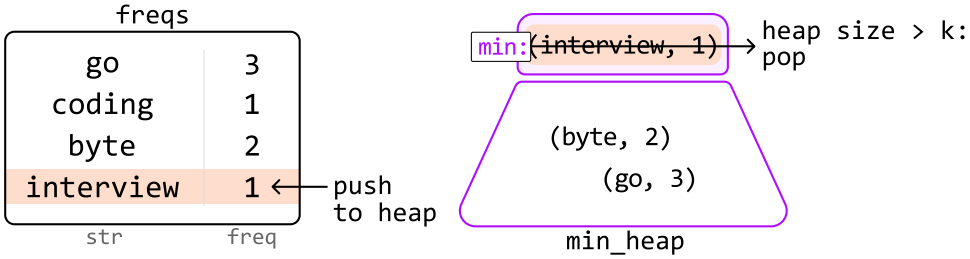
In the previous approach, we ended up storing up to n items in the heap. However, since we only need the k most frequent characters, is there a way to maintain a heap with a space complexity of $O(k)$?

An important observation is that when our heap exceeds size k, we can **discard the lowest frequency strings until the heap's size is reduced to k again**. We can do this because those discarded strings definitely won't be among the k most frequent strings.

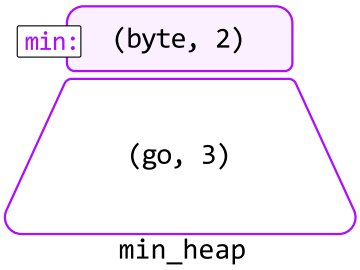
However, we can't implement this strategy with a max-heap because we won't have access to the lowest frequency string. Instead, we need to use a **min-heap**.

Let's observe how this works over an example:

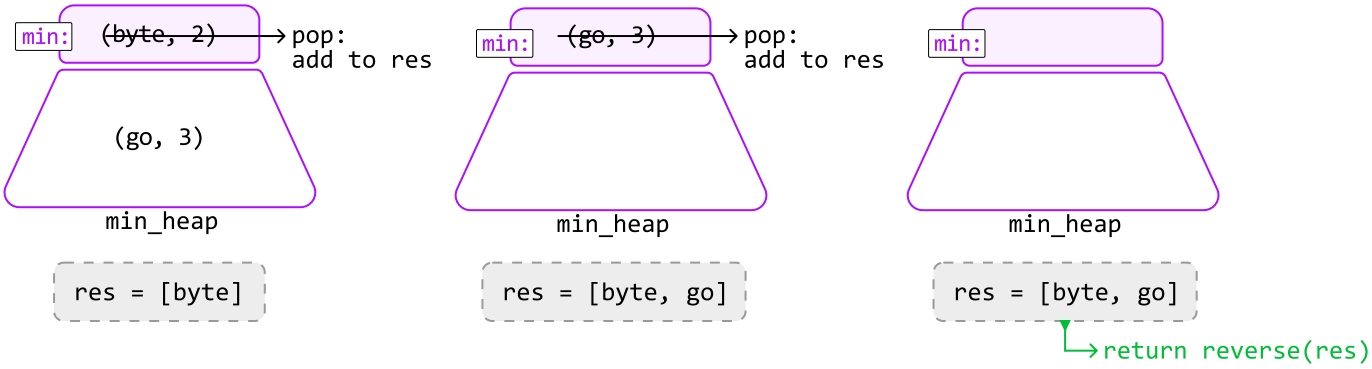




In the end, the strings remaining in the heap are our top k frequent strings:



To retrieve these strings, pop them from the heap until it's empty. Because we're using a min-heap, we're popping off the less frequent strings first. So, we need to reverse the order of the retrieved strings before returning the result:



Python Java

```
import java.util.ArrayList;
import java.util.HashMap;
import java.util.Map;
import java.util.PriorityQueue;

class Pair {
    String str;
    int freq;

    public Pair(String str, int freq) {
        this.str = str;
        this.freq = freq;
    }
}
```

```
public class Main {
    public ArrayList<String> k_most_frequent_strin
        // Count the frequency of each string.
        Map<String, Integer> freqs = new HashMap<>
        for (String s : strs) {
            freqs.put(s, freqs.getOrDefault(s, 0)
        }
        // Min-heap with a custom comparator: lowe
        PriorityQueue<Pair> minHeap = new Priority
            // If frequencies are equal, prioritiz
            if (a.freq == b.freq) {
                return b.str.compareTo(a.str);
            }
            return a.freq - b.freq;
        });
        // Maintain a heap of size 'k' with the mo
        for (Map.Entry<String, Integer> entry : fr
            minHeap.offer(new Pair(entry.getKey(),
            if (minHeap.size() > k) {
                minHeap.poll();
            }
        }
        // Pop elements from the heap and reverse
        ArrayList<String> res = new ArrayList<>();
        while (!minHeap.isEmpty()) {
            res.add(minHeap.poll().str);
        }
        // Reverse the list since it's a min-heap
        ArrayList<String> reversed = new ArrayList
        for (int i = res.size() - 1; i >= 0; i--)
            reversed.add(res.get(i));
        }
        return reversed;
    }
}
```

Complexity Analysis

Time complexity: The time complexity of `k_most_frequent_strings_min_heap` is $O(n \log(k))$.

- It takes $O(n)$ time to count the frequency of each string using `Counter`.
- To populate the heap, we push n words onto it, with each `push` and `pop` operation taking $O(\log(k))$ time. This takes $O(n \log(k))$ time.
- Then, we extract k strings from the heap by performing the `pop` operation k times. This takes $O(k \log(k))$ time.

- Finally, we reverse the output array, which takes $O(k)$ time.

Therefore, the overall time complexity is $O(n) + O(n \log(k)) + O(k \log(k)) + O(k) = O(n \log(k))$.

Space complexity: The space complexity is $O(n)$ because the hash map stores at most n pairs, whereas the heap only takes up $O(k)$ space. The `res` array is not considered in the space complexity.